Setting up your environment

Outline

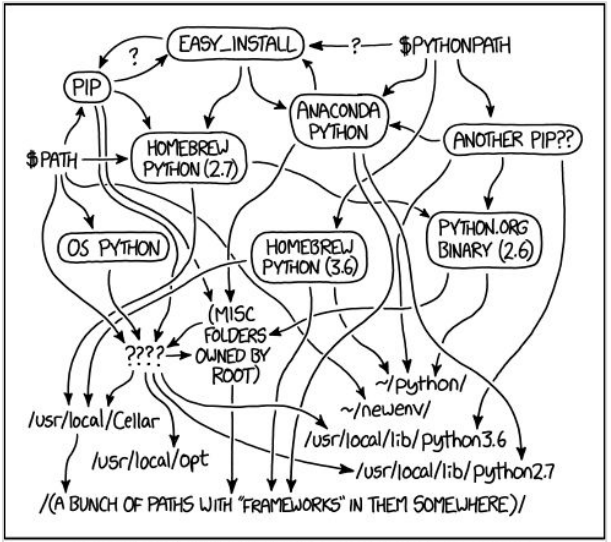
- uv installation
- Cloning training repo
- Installing dependencies in an isolated environment
- Setting up VS code

Before You Start (Prerequisites)

- A terminal (macOS Terminal, Linux shell, Windows Subsystem for Linux)
- Git installed (`git --version` to check)
- Internet connection
- A code editor (e.g., VS Code)

The Problem with Python Environments

- Python setups are messy: different Python versions, conflicting packages, “works on my machine”
- Virtual environments isolate each project’s dependencies
- Traditionally this took several tools (pip, venv, pyenv, pip-tools...)



https://xkcd.com/1987/

What is `uv`?

- A fast Python package and project manager, made by Astral
- Handles: Python versions, virtual environments, dependency installation, lockfiles, running scripts
- “One tool instead of many - and much faster”

Advantages of `uv`

- **Speed:** typically much faster than pip
- **Reproducibility:** lockfile (`uv.lock`) ensures everyone gets identical versions
- **Simplicity:** one command (`uv sync`) sets up everything
- **Manages Python itself:** installs the right Python version if needed
- **Isolation by default:** project-specific `.venv`, no polluting your system Python

Installing `uv`

- MacOS/Linux Terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- Alternatives: `brew install uv`, `pip install uv`
- Verify: `uv --version`
- Installation guide: https://docs.astral.sh/uv/getting-started/installation/

Cloning the Repo

- What cloning does: copies the repo (code + config files) to your machine
- Command:

```
git clone
https://github.com/uchicago-dsi/national-lab-agentic-training.git
```

What's in the Repo?

- Python Environment
  - `pyproject.toml`: declares the project and its dependencies
  - `uv.lock`: exact pinned versions for reproducibility
- Other files and folders to be used during the training (slides and activities)

Running `uv sync`

- Command: `uv sync`
- What it does:
  1. Finds or installs the right Python version
  2. Creates `.venv`, which holds the isolated environment
  3. Installs exact dependencies from `uv.lock`

Setting Up VS Code

- Install the **Python extension** (Microsoft)
- **Open the repo folder**, not a single file: File → Open Folder
- Open the Command Palette: `Cmd+Shift+P` (Mac) or `Ctrl+Shift+P` (Windows/Linux)
- Type **“Python: Select Interpreter”**
- Choose the entry that points to **`.venv`**

`uv` Command Cheat Sheet

Command	What it does
`uv sync`	Install dependencies from the lockfile
`uv run <cmd>`	Run a command inside the project environment
`uv add <package>`	Add a dependency to the project
`uv remove <package>`	Remove a dependency
`uv python install 3.12`	Install a specific Python version
`uv run python <script>.py`	Run a script in the project environment
`uv run --env-file .env <script>`	Run a script with variables loaded from `.env`

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Next Steps

- Complete setup *before* the session, bring your laptop charged
- Make sure you can access a coding agent and connect to a model using the ALCF Inference Service (video 4)
- See you at the training!